

Arrays – introduction to arrays, importing array module, creating arrays, accessing and modifying elements, and operations.

An array is a collection of items of the same type stored at contiguous memory locations. Python provides the array module to work with arrays.

1. Introduction to Arrays

- Arrays store multiple items of the same data type.
- Elements are stored in contiguous memory locations.
- Arrays are more efficient than lists for large numeric data.

2. Importing array module

```
import array as arr    # import array module
```

3. Creating Arrays

Arrays are created using `array(typecode, initializer)`.

```
import array as arr
a = arr.array('i')           # empty integer array
b = arr.array('i', [10, 20, 30, 40]) # integer array
c = arr.array('f', [1.5, 2.5, 3.5]) # float array
d = arr.array('u', 'hello')   # unicode array
```

Typecodes: 'i' → signed int 'f' → float 'u' → unicode

4. Accessing Elements

Elements are accessed using index (0-based indexing).

```
b = arr.array('i', [10, 20, 30, 40])
print(b[0])    # 10
print(b[2])    # 30
print(b[-1])   # 40 (last element)
```

5. Modifying Elements

Elements can be changed using index.

```
b = arr.array('i', [10, 20, 30, 40])
b[1] = 99      # modify element at index 1
b[3] = -10     # modify element at index 3
print(b)       # array('i', [10, 99, 30, -10])
```

6. Basic Operations

Arrays support various operations.

```
a = arr.array('i', [10, 20, 30])
b = arr.array('i', [40, 50])

print(len(a))           # length of array
print(a.buffer_info())  # (address, length in bytes)
print(a.typecode)       # type code
print(a.tolist())       # convert array to list
a.append(40)            # add element at the end
a.extend(b)             # add elements from another array
a.insert(1, 15)         # insert 15 at index 1
a.remove(20)            # remove first occurrence of 20
a.pop()                # remove and return last element
a.pop(2)               # remove and return element at index 2
a.reverse()            # reverse the array
a.count(30)            # count occurrences of 30
print(a)
```

7. Arithmetic Operations (element-wise)

Arithmetic operations are performed element-wise.

```
x = arr.array('i', [1, 2, 3])
y = arr.array('i', [10, 20, 30])
z = arr.array('i')
for i in range(len(x)):
    z.append(x[i] + y[i]) # addition
print(z)                 # array('i', [11, 22, 33])
```

8. Other Operations

```
a = arr.array('i', [5, 1, 3, 2])
a.sort()           # sort the array
print(a)          # array('i', [1, 2, 3, 5])
a.index(3)         # return index of first occurrence of 3
a.count(2)         # count occurrences of 2
a.clear()          # remove all elements
print(a)          # array('i') empty array
```

Key Points

- Arrays store elements of same data type.
- Use `array()` constructor with typecode to create arrays.
- Elements are accessed and modified using index.
- Various operations like `append()`, `insert()`, `remove()`, `pop()`, `extend()`, `reverse()`, `sort()` etc. are supported.
- Arrays are memory efficient and faster for numeric data.

Arrays – introduction to arrays

An array is a collection of items of the same type stored at contiguous memory locations. Python provides the array module to work with arrays.

1. Introduction to Arrays

- Arrays store multiple items of the same data type.
- Elements are stored in contiguous memory locations.
- Arrays are more efficient than lists for large numeric data.
- In an array, all elements are of the same type and size.
- Arrays are fixed in size after creation.

Example : An array of integers → `arr = [ 10 , 20 , 30 , 40 , 50 ]`

	↑	↑	↑	↑	↑
index	0	1	2	3	4

Arrays – importing array module

In Python, arrays are not built-in like lists.

To use arrays, we need to import the **array module**.

This module provides the array object which allows us to store multiple items of the same data type.

1. Importing array module

We import the array module using the import statement.

```
import array as arr      # import array module
```

Example : Import the array module.

```
import array as arr
```

```
# now we can use arr.array() to  
# create and work with arrays.
```

Key Point :

We must import the array module before creating an array.

Arrays - creating arrays

After importing the array module, we can create arrays using the `array()` constructor.

1. Creating Arrays

We create arrays by specifying the type code and an initializer.

```
arr = array(typecode, initializer)    # typecode is one of 'i', 'f', 'u' etc.
```

Where : typecode → type of elements to be stored in the array.

initializer → a list / tuple / string / another array used to initialize.

Examples :

```
import array as arr          # import array module
a = arr.array('i')           # create empty integer array
b = arr.array('i', [10, 20, 30, 40]) # integer array with initializer
c = arr.array('f', [1.5, 2.5, 3.5]) # float array
d = arr.array('u', 'hello')   # unicode array (characters)
```

Output / Result :

```
a → array('i')
b → array('i', [10, 20, 30, 40])
c → array('f', [1.5, 2.5, 3.5])
d → array('u', 'hello')
```

Key Point :

- We must specify the correct typecode according to the type of data we want to store in the array.

Arrays – accessing and modifying elements

Elements in an array are accessed and modified using their index.
Indexing in arrays is 0-based (first element is at index 0).

1. Accessing Elements

We can access elements of an array using the index inside square brackets.

```
import array as arr          # import array module
a = arr.array('i', [10, 20, 30, 40, 50]) # integer array
print(a[0])                  # 10
print(a[2])                  # 30
print(a[4])                  # 50
```

Note : If we try to access an index that does not exist, Python raises an `IndexError`.

Example :

```
a = arr.array('i', [10, 20, 30, 40])
print(a[1])      # Output: 20
print(a[4])      # IndexError: array index out of range
                 # (valid index : 0 to 3)
```

2. Modifying Elements

We can change the value of an existing element by assigning a new value to that index.

```
import array as arr          # import array module
a = arr.array('i', [10, 20, 30, 40, 50])
a[2] = 300                  # change element at index 2
a[4] = 500                  # change element at index 4
print(a)                   # Output: array('i', [10, 20, 300, 40, 500])
```

Example :

```
a = arr.array('i', [5, 15, 25, 35])
a[1] = 150                  # modify element at index 1
print(a)                   # Output: array('i', [5, 150, 25, 35])
a[3] = 350                  # modify element at index 3
print(a)                   # Output: array('i', [5, 150, 25, 350])
```

Key Points :

- Use `a[index]` to access element at index.
- Use `a[index] = value` to modify element at index.
- Index starts from 0.
- Accessing or modifying invalid index raises `IndexError`.

Arrays - operations.

Python arrays support various operations to perform common tasks.

Some of the commonly used operations are :

1. Traversing / Iterating

We can traverse array elements using a loop.

```
import array as arr
a = arr.array('i', [10, 20, 30, 40])
for x in a:
    print(x)                                # Output: 10 20 30 40
```

3. Adding Elements

Use `append()` to add an element at the end.

```
import array as arr
a = arr.array('i', [10, 20, 30])
a.append(40)
print(a)                                # Output: array('i', [10, 20, 30, 40])
```

5. Searching Elements

Use `index()` to find the index of an element.

```
import array as arr
a = arr.array('i', [10, 20, 30, 40])
print(a.index(30))                        # Output: 2
print(a.index(50))                        # Output: ValueError (not found)
```

7. Removing Elements

Use `remove()` to remove the first occurrence of an element.

```
import array as arr
a = arr.array('i', [10, 20, 30, 20])
a.remove(20)                             # removes first 20
print(a)                                # Output: array('i', [10, 30, 20])
```

2. Length of Array

Use `len()` to get the number of elements.

```
import array as arr
a = arr.array('i', [10, 20, 30, 40])
print(len(a))                            # Output: 4
```

4. Extending Array

Use `extend()` to add elements from another iterable.

```
import array as arr
a = arr.array('i', [10, 20])
a.extend([30, 40, 50])
print(a)                                # Output: array('i', [10, 20, 30, 40, 50])
```

6. Counting Occurrences

Use `count()` to count how many times an element occurs.

```
import array as arr
a = arr.array('i', [10, 20, 20, 30, 20, 40])
print(a.count(20))                       # Output: 3
```

8. Reversing Array

Use `reverse()` to reverse the order of elements.

```
import array as arr
a = arr.array('i', [10, 20, 30, 40])
a.reverse()
print(a)                                # Output: array('i', [40, 30, 20, 10])
```

Key Points :

- Arrays support many useful operations.
- Use built-in methods to perform tasks efficiently.
- Always handle errors while searching or removing elements.